

# **CheetahKEM: Algorithm Specifications And Supporting Documentation**

Chengdong Tao

June 27, 2026

# Contents

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                    | <b>3</b>  |
| <b>2</b> | <b>Algorithm Description</b>                                           | <b>4</b>  |
| 2.1      | Mathematical Preliminaries . . . . .                                   | 4         |
| 2.2      | Cheetah Public-Key Encryption Algorithm . . . . .                      | 13        |
| 2.3      | Cheetah Key Encapsulation Mechanism . . . . .                          | 17        |
| 2.4      | Parameters . . . . .                                                   | 18        |
| <b>3</b> | <b>Design Principles</b>                                               | <b>19</b> |
| 3.1      | The RLWE, MLWE and LWE problem . . . . .                               | 19        |
| 3.2      | Noise Distribution . . . . .                                           | 19        |
| <b>4</b> | <b>Performance Analysis</b>                                            | <b>20</b> |
| 4.1      | Performance . . . . .                                                  | 20        |
| 4.2      | Test Vector . . . . .                                                  | 21        |
| <b>5</b> | <b>Security Analysis</b>                                               | <b>22</b> |
| 5.1      | Security Proof . . . . .                                               | 22        |
| 5.2      | Complexity of the BKZ Algorithm . . . . .                              | 23        |
| 5.3      | Practical Security Evaluation of the Cheetah KEM . . . . .             | 23        |
| 5.3.1    | Primal Attack . . . . .                                                | 24        |
| 5.3.2    | Dual Attack . . . . .                                                  | 24        |
| 5.3.3    | Attack Complexity of the Cheetah Key Encapsulation Mechanism . . . . . | 25        |
| <b>6</b> | <b>Advantages and Disadvantages</b>                                    | <b>25</b> |

# 1 Introduction

This submission presents the basic principle and implementation of the Cheetah key encapsulation mechanism (CheetahKEM for short). The CheetahKEM is a family of lattice-based cryptographic algorithms whose security base on the hardness of the Ring Learning With Errors problem (RLWE for short) and Module Learning With Errors (MLWE for short) problem. Specifically, Cheetah128 is a key encapsulation mechanism based on the RLWE problem, while Cheetah256, Cheetah384, and Cheetah512 are key encapsulation mechanisms based on the MLWE problem. We first construct a CPA-secure public-key encryption scheme, and then use the Fujisaki-Okamoto transformation to build an IND-CCA2-secure key encapsulation mechanism.

We briefly describe the basic principle of the CPA-secure Cheetah public-key encryption algorithm. For simplicity, we omit the compression and decompression steps for public keys and ciphertexts, as well as the NTT optimization process. Let  $N = 640$ ,  $q = 7681$ ,  $k$  be a positive integer, and  $\lambda < N$  be the security parameter. Let  $R_q = \mathbb{Z}_q[X]/X^N + 1$ . The key generation steps of the Cheetah public-key encryption algorithm are as follows:

1. Randomly generate a polynomial matrix  $\mathbf{A} \in R_q^{k \times k}$ .
2. Randomly generate noise polynomial vectors  $\mathbf{s} \in R_q^k$  and  $\mathbf{e} \in R_q^k$ , whose polynomial coefficients follow the centered binomial distribution  $\psi_\eta$ .
3. Compute  $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$ .
4. The public key is  $\mathbf{pk} = (\mathbf{A}, \mathbf{b})$ , and the secret key is  $\mathbf{sk} = \mathbf{s}$ .

Let  $\mathbf{m} \in \{0, 1\}^\lambda$  be a plaintext bit string. We first represent the plaintext as a polynomial  $\mathbf{m} = m_0 + m_1X + \dots + m_{\lambda-1}X^{\lambda-1} \in R_q$ . Let  $\Delta = 2^{-1} \bmod q$ . The encryption steps of the Cheetah public-key encryption algorithm are as follows:

1. Randomly generate noise polynomial vectors  $\mathbf{e}_1 \in R_q^k$ ,  $\mathbf{e}_2 \in R_q^k$  and  $\mathbf{e}_3 = e_{3,0} + e_{3,1}X + e_{3,\lambda-1}X^{\lambda-1} \in R_q$ , whose coefficients follow the centered binomial distribution  $\psi_\eta$ .
2. Compute  $\mathbf{c}_1^\top = \mathbf{e}_1^\top \mathbf{A} + \mathbf{e}_2^\top$ .
3. Compute  $\mathbf{c}_2 = \text{Truncate}(\mathbf{e}_1^\top \mathbf{b}) + \mathbf{e}_3 + \Delta \cdot \mathbf{m}$ , where the **Truncate** function is referred to Section 2.1. The ciphertext  $\mathbf{c}_2$  has only  $\lambda$  coefficients, which reduces the ciphertext length.
4. The ciphertext is  $(\mathbf{c}_1, \mathbf{c}_2)$ .

The decryption steps of the Cheetah public-key encryption algorithm are as follows:

1. Compute  $\mathbf{u} = 2(\mathbf{c}_2 - \text{Truncate}(\mathbf{c}_1^\top \mathbf{s})) \bmod \pm q$ .
2. Treat  $\mathbf{u}$  as an integer polynomial and compute  $\mathbf{m} = \mathbf{u} \bmod 2$ , then  $\mathbf{m}$  is the plaintext.

In previous LWE-based encryption schemes, the last step of decryption is usually to compare the ciphertext with  $q/4$ : if it is larger than  $q/4$ , the corresponding plaintext bit is 1; otherwise, it is 0. In the Cheetah encryption algorithm, we only need to perform a modular operation to recover the plaintext. This is similar to the decryption process of the NTRU encryption algorithm and improves the decryption speed.

We summarize the Cheetah public-key encryption algorithm in Table 1.

The rest of this paper is organized as follows. Section 2 describes the Cheetah key encapsulation mechanism and its parameters in detail. Section 3 introduces the design principle of the Cheetah key encapsulation mechanism. Section 4 presents the implementation and performance of the Cheetah key encapsulation mechanism. Section 5 analyzes the security of the Cheetah key encapsulation mechanism. Section 6 discusses the advantages and disadvantages of the Cheetah key encapsulation mechanism.

---

**KeyGen:**

- 1:  $\mathbf{A} \leftarrow R_q^{k \times k}, \mathbf{s} \in R_q^k \leftarrow \psi_\eta, \mathbf{e} \in R_q^k \leftarrow \psi_\eta$
- 2:  $\mathbf{b} \leftarrow \mathbf{A}\mathbf{s} + \mathbf{e}$
- 3: **return**  $\text{pk} = (\mathbf{A}, \mathbf{b}), \text{sk} = \mathbf{s}$

**Encrypt(pk, m):**

- 1:  $\mathbf{e}_1 \in R_q^k \leftarrow \psi_\eta, \mathbf{e}_2 \in R_q^k \leftarrow \psi_\eta, \mathbf{e}_3 \in R_q \leftarrow \psi_\eta$
- 2:  $\mathbf{c}_1^\top = \mathbf{e}_1^\top \mathbf{A} + \mathbf{e}_2^\top$
- 3:  $\mathbf{c}_2 = \text{Truncate}(\mathbf{e}_1^\top \mathbf{b}) + \mathbf{e}_3 + \Delta \cdot \mathbf{m}$
- 4: **return**  $(\mathbf{c}_1, \mathbf{c}_2)$

**Decrypt(sk, c<sub>1</sub>, c<sub>2</sub>):**

- 1:  $\mathbf{u} \leftarrow 2(\mathbf{c}_2 - \text{Truncate}(\mathbf{c}_1^\top \mathbf{s})) \bmod \pm q$
  - 2:  $\mathbf{m} \leftarrow \mathbf{u} \bmod 2$
  - 3: **return**  $\mathbf{m}$
- 

Table 1: Cheetah public key encryption algorithm

## 2 Algorithm Description

### 2.1 Mathematical Preliminaries

**Integer Modular Arithmetic** Let  $\mathbb{Z}$  be the ring of integers,  $q$  a prime, and  $\mathbb{Z}_q$  the quotient ring  $\mathbb{Z}/q\mathbb{Z}$ . For an integer  $a$ , the modular operation  $r = a$

$\bmod q$  means  $r \in \{0, 1, \dots, q-1\}$ , while the modular operation  $r = a \bmod^{\pm} q$  means  $r \in \{-\frac{q-1}{2}, \dots, -1, 0, 1, \dots, \frac{q-1}{2}\}$ . Denote  $d_q = \lceil \log_2(q) \rceil$ .

**Norms** Let  $a \in \mathbb{Z}_q$  and  $\mathbf{a} = (a_0, a_1, \dots, a_{n-1}) \in \mathbb{Z}_q^n$ . We define

$$\|a\|_{\infty} = |a \bmod^{\pm} q|, \quad \|\mathbf{a}\|_{\infty} = \max_{0 \leq i \leq n-1} \|a_i\|_{\infty}.$$

$$\|\mathbf{a}\|_2 = (\|a_0\|_{\infty}^2 + \dots + \|a_{n-1}\|_{\infty}^2)^{1/2}.$$

**Compression Function** Let  $d$  be a positive even integer and  $x \in \{0, 1, \dots, q-1\}$ . We define the compression function as follows:

$$\text{Compress}(x, d) = \left\lfloor \frac{x}{d} \right\rfloor.$$

Let  $y = \text{Compress}(x, d)$ . We define the decompression function as follows:

$$\text{Decompress}(y, d) = \min \left( y \cdot d + \frac{d}{2}, q-1 \right).$$

When  $d$  is a power of two, compression and decompression of elements in  $\mathbb{Z}_q$  can be implemented using only shift and addition operations, without division or modular operations.

Let  $\tilde{x} = \text{Decompress}(\text{Compress}(x, d), d)$ . For any  $x \in \mathbb{Z}_q$ , it is easy to show that  $|x - \tilde{x}| \leq \frac{d}{2}$ . Indeed, by the definition of the floor function, there exists an integer  $r \in [0, d)$  such that  $x = yd + r$ . We prove this in two cases. If  $yd + \frac{d}{2} \leq q-1$ , then  $|x - \tilde{x}| = |r - \frac{d}{2}| \leq \frac{d}{2}$ . If  $yd + \frac{d}{2} > q-1$ , then  $\tilde{x} = q-1$ . On the other hand,  $x \geq yd > (q-1) - \frac{d}{2}$  (otherwise  $yd + \frac{d}{2} \leq q-1$ ). Thus  $|x - \tilde{x}| = (q-1) - x < \frac{d}{2}$ .

Let  $\mathbf{a} \in \mathbb{Z}_q^n$  and  $\mathbf{A} \in \mathbb{Z}_q^{n \times n}$ . Then  $\text{Compress}(\mathbf{a}, d)$  and  $\text{Compress}(\mathbf{A}, d)$  denote the component-wise compression of the vector  $\mathbf{a}$  and matrix  $\mathbf{A}$ , respectively. Similarly,  $\text{Decompress}(\mathbf{a}, d)$  and  $\text{Decompress}(\mathbf{A}, d)$  denote the component-wise decompression of the vector  $\mathbf{a}$  and matrix  $\mathbf{A}$ , respectively.

**Bit Strings and Byte Arrays** Let  $\mathbb{B} = \{0, 1, 2, \dots, 255\}$  and  $t$  be a positive integer. Algorithm 1 converts a bit array  $\mathbf{b}$  into a byte array **bytes**. Given a bit string  $b_0b_1b_2b_3b_4b_5b_6b_7$ , the corresponding byte is  $b_0 + b_1 \cdot 2 + b_2 \cdot 2^2 + \dots + b_7 \cdot 2^7$ , that is, we use the least significant bit (LSB) first order. For example, the bit string 11111011 corresponds to the byte  $2^0 + 2^1 + 2^2 + 2^3 + 2^4 + 2^6 + 2^7 = 223$ .

---

**Algorithm 1** BitsToBytes(**b**): Bit array to byte array conversion.

---

**Input:** Bit array  $\mathbf{b} \in \{0, 1\}^{8t}$ .

**Output:** Byte array  $\mathbf{bytes} \in \mathbb{B}^t$ .

```

1: bytes  $\leftarrow (0, \dots, 0)$ 
2: for ( $i = 0; i < t; i++$ ) do
3:    $\mathbf{bytes}[i] \leftarrow \sum_{j=0}^7 \mathbf{b}[8i + j] \cdot 2^j$ 
4: end for
5: return bytes

```

---

Algorithm 2 converts a byte array **bytes** into a bit array **b**. If an element of **bytes** is  $b_0 + b_1 \cdot 2 + \dots + b_7 \cdot 2^7$ , then its corresponding bit string is  $b_0 b_1 \dots b_7$ .

---

**Algorithm 2** BytesToBits(**bytes**): Byte array to bit array conversion.

---

**Input:** Byte array  $\mathbf{bytes} \in \mathbb{B}^t$ .

**Output:** Bit array  $\mathbf{b} \in \{0, 1\}^{8t}$ .

```

1: c  $\leftarrow \mathbf{bytes}$ 
2: for ( $i = 0; i < t; i++$ ) do
3:   for ( $j = 0; j < 8; j++$ ) do
4:      $\mathbf{b}[8i + j] \leftarrow \mathbf{c}[i] \bmod 2$ 
5:      $\mathbf{c}[i] \leftarrow \lfloor \mathbf{c}[i] / 2 \rfloor$ 
6:   end for
7: end for
8: return b

```

---

**Integer Arrays and Byte Arrays** Let  $d$  be a positive integer. Algorithm 3 converts an integer array into a byte array, where the elements of the integer array are defined in the interval  $[0, 2^d - 1]$  and the length of the array is  $n$ . In the Cheetah key encapsulation mechanism,  $n$  is a multiple of 8. Let an element of the integer array **c** be  $b_0 + b_1 \cdot 2 + \dots + b_{d-1} \cdot 2^{d-1}$ , then its corresponding bit string is  $b_0 b_1 \dots b_{d-1}$ . The function **IntegersToBytes** first converts the integer array  $c_0, c_1, \dots, c_{n-1}$  of length  $n$  into a bit string  $b_0, b_1, \dots, b_{nd-1}$ , and then uses the **BitsToBytes** function to convert the bit string into a byte array.

---

**Algorithm 3** `IntegersToBytes(c, n, d)`: Integer array to byte array conversion.

---

**Input:** Integer array  $\mathbf{c} \in \mathbb{Z}_{2^d}^n$ , array length  $n$ , positive integer  $d$ .

**Output:** Byte array  $\mathbf{bytes} \in \mathbb{B}^{nd/8}$ .

```

1:  $\mathbf{b} \leftarrow (0, 0, \dots, 0)$  ▷ Length of  $\mathbf{b}$  is  $nd$ .
2: for ( $i = 0; i < n; i++$ ) do
3:    $a \leftarrow \mathbf{c}_i$ 
4:   for ( $j = 0; j < d; j++$ ) do
5:      $\mathbf{b}[i \cdot d + j] \leftarrow a \bmod 2$ 
6:      $a \leftarrow (a - \mathbf{b}[i \cdot d + j]) / 2$ 
7:   end for
8: end for
9:  $\mathbf{bytes} \leftarrow \text{BitsToBytes}(\mathbf{b})$ 
10: return  $\mathbf{bytes}$ 

```

---

Algorithm 4 converts a byte array into an integer array. First, the `BytesToBits` function is used to convert the byte array into a bit string in little-endian (LSB-first) order. The bit string is then converted into an integer array.

---

**Algorithm 4** `BytesToIntegers(bytes, n, d)`: Byte array to integer array conversion.

---

**Input:** Byte array  $\mathbf{bytes} \in \mathbb{B}^{nd/8}$ , positive integers  $n$  and  $d$ .

**Output:** Integer array  $\mathbf{c} \in \mathbb{Z}_{2^d}^n$ .

```

1:  $\mathbf{b} \leftarrow \text{BytesToBits}(\mathbf{bytes})$ 
2: for ( $i = 0; i < n; i++$ ) do
3:    $\mathbf{c}[i] \leftarrow \sum_{j=0}^{d-1} \mathbf{b}[i \cdot d + j] \cdot 2^j \bmod 2^d$ 
4: end for
5: return  $\mathbf{c}$ 

```

---

**Polynomial Rings** Let  $N = 640$  and  $q = 7681$ . We define the polynomial rings  $\mathcal{R} = \mathbb{Z}[X]/(X^N + 1)$  and  $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$ . The coefficients of polynomials in  $\mathcal{R}$  and  $\mathcal{R}_q$  are defined over  $\mathbb{Z}$  and  $\mathbb{Z}_q$ , respectively. We use bold lowercase letters to denote elements of  $\mathcal{R}_q$ . For any  $\mathbf{f}(X) = f_0 + f_1X + \dots + f_{N-1}X^{N-1} \in \mathcal{R}_q$ , we usually write it as  $\mathbf{f} = (f_0, f_1, \dots, f_{N-1})$ . Without ambiguity, bold lowercase letters sometimes represent polynomials and sometimes represent coefficient vectors of polynomials. Let  $\eta$  be a positive integer with  $\eta < q$ , and let  $\mathbf{e} \in \mathcal{R}_q$ . If all coefficients of  $\mathbf{e}$  lie in the set  $\{-\eta, \dots, -1, 0, 1, \dots, \eta\}$ , we call  $\mathbf{e}$  a noise polynomial or error polynomial. Let  $\mathbf{m} \in \mathcal{R}_q$ . If all coefficients of  $\mathbf{m}$  lie in the set  $\{0, 1\}$ , we call  $\mathbf{m}$  a binary polynomial. Let  $\mathbf{v} = v_0 + v_1X + \dots + v_{N-1}X^{N-1} \in \mathcal{R}_q$  and let  $\lambda$  be a positive integer with  $\lambda < N$ . We define the truncated polynomial of  $\mathbf{v}$  as follows:

$$\text{Truncate}(\mathbf{v}, \lambda) = v_0 + v_1X + \dots + v_{\lambda-1}X^{\lambda-1}.$$

The main operations of the Cheetah key encapsulation mechanism are polynomial addition and multiplication over  $\mathcal{R}_q$ , where the multiplication can be implemented using the NTT transform.

**NTT Transformation** Let  $\psi$  be a primitive  $2N$ -th root of unity in  $\mathbb{Z}_q$ , and  $\omega = \psi^2$  be a primitive  $N$ -th root of unity in  $\mathbb{Z}_q$ . Let  $\mathbf{a} = a_0 + a_1X + \dots + a_{N-1}X^{N-1} \in R_q$ . The Number Theoretic Transform of the polynomial  $\mathbf{a}$  is defined as

$$\text{NTT}(\mathbf{a}) = \hat{\mathbf{a}} = \hat{a}_0 + \hat{a}_1X + \dots + \hat{a}_{N-1}X^{N-1},$$

where  $\hat{a}_i = \sum_{j=0}^{N-1} \psi^{j(2i+1)} a_j \pmod{q}$  for  $i = 0, 1, \dots, N-1$ . The inverse Number Theoretic Transform of the vector  $\hat{\mathbf{a}}$  is defined as

$$\text{INTT}(\hat{\mathbf{a}}) = \mathbf{a} = a_0 + a_1X + \dots + a_{N-1}X^{N-1},$$

where  $a_i = N^{-1} \sum_{j=0}^{N-1} \psi^{-i(2j+1)} \hat{a}_j \pmod{q}$  for  $i = 0, 1, \dots, N-1$ .

In the Cheetah key encapsulation mechanism,  $q = 7681$  and  $N = 640 = 128 \times 5$ . Let  $N_1 = 128$  and  $N_2 = 5$ . We use the Good-Thomas algorithm to compute  $\text{NTT}_{640}$  and  $\text{INTT}_{640}$  efficiently. The detailed procedure is given in Algorithm 5. The inputs of Algorithm 5 are the vector  $\mathbf{a}$  and *dir*. When *dir* = FORWARD, the output is  $\mathbf{z} = \text{NTT}_{640}(\mathbf{a})$ . When *dir* = INVERSE, the output is  $\mathbf{z} = \text{INTT}_{640}(\mathbf{a})$ .

Lines 6-10 of the algorithm reshape the vector  $\mathbf{a}$  into a 2-dimensional array with  $N_1$  rows and  $N_2$  columns. Lines 11-13 perform the  $\text{NTT}_{128}$  transform on each column of the 2-dimensional array. Lines 14-16 perform the  $\text{NTT}_5$  transform on each row. Lines 17-21 convert the 2-dimensional array back into a 1-dimensional vector.

The  $\text{NTT}_5$  transform is shown in Algorithm 6 and can be implemented via hardcoding. The  $\text{NTT}_{128}$  transform is shown in Algorithm 7, which is implemented using the Cooley-Tukey algorithm. The `bit_reverse` function in Algorithm 7 is defined as follows: let  $z$  be a positive integer with binary representation  $z = z_d \cdot 2^d + \dots + z_1 \cdot 2 + z_0$ , where  $z_j \in \{0, 1\}$  for  $j = 0, 1, \dots, d$ . Then

$$\text{bit\_reverse}(z) = z_0 \cdot 2^d + \dots + z_{d-1} \cdot 2 + z_d.$$

---

**Algorithm 5**  $\text{NTT}_{640}(\mathbf{a}, \text{dir})$ 

---

**Input:**  $\mathbf{a} \in \mathbb{Z}_q^N$  and  $\text{dir} \in \{\text{FORWARD}, \text{INVERSE}\}$ .

**Output:**  $\mathbf{z}$ .

```
1: if dir == FORWARD then
2:   for  $i = 0$  to  $N - 1$  do
3:      $\mathbf{a}[i] \leftarrow \mathbf{a}[i] \cdot \psi^i \bmod q$ 
4:   end for
5: end if
6: for  $n_1 = 0$  to  $N_1 - 1$  do
7:   for  $n_2 = 0$  to  $N_2 - 1$  do
8:      $\mathbf{A}[n_1, n_2] \leftarrow \mathbf{a}[(n_1 \cdot N_2 + n_2 \cdot N_1) \bmod N]$ 
9:   end for
10: end for
11: for  $n_2 = 0$  to  $N_2 - 1$  do
12:    $\tilde{\mathbf{A}}[\cdot][n_2] \leftarrow \text{NTT}_{128}(\mathbf{A}[\cdot][n_2], \text{dir})$ 
13: end for
14: for  $n_1 = 0$  to  $N_1 - 1$  do
15:    $\hat{\mathbf{A}}[n_1][\cdot] \leftarrow \text{NTT}_5(\tilde{\mathbf{A}}[n_1][\cdot], \text{dir})$ 
16: end for
17: for  $n_1 = 0$  to  $N_1 - 1$  do
18:   for  $n_2 = 0$  to  $N_2 - 1$  do
19:      $\mathbf{z}[(n_1 \cdot N_2 + n_2 \cdot N_1) \bmod N] \leftarrow \hat{\mathbf{A}}[n_1, n_2]$ 
20:   end for
21: end for
22: if dir == INVERSE then
23:   for  $i = 0$  to  $N - 1$  do
24:      $\mathbf{z}[i] \leftarrow \mathbf{z}[i] \cdot N^{-1} \cdot \psi^{-i} \bmod q$ 
25:   end for
26: end if
27: return  $\mathbf{z}$ 
```

---

---

**Algorithm 6**  $\text{NTT}_5(\mathbf{a}, \text{dir})$ 

---

**Input:**  $\mathbf{a} = (a_0, a_1, a_2, a_3, a_4) \in \mathbb{Z}_q^5$  and  $\text{dir} \in \{\text{FORWARD}, \text{INVERSE}\}$ .

**Output:**  $\mathbf{z} = (z_0, z_1, z_2, z_3, z_4)$ .

```
1: If dir == FORWARD then  $\theta \leftarrow \omega_5$ , else  $\theta \leftarrow \omega_5^{-1}$   $\triangleright \omega_5^5 \bmod q = 1$ .
2:  $z_0 \leftarrow (a_0 + a_1 + a_2 + a_3 + a_4) \bmod q$ 
3:  $z_1 \leftarrow (a_0 + a_1\theta + a_2\theta^2 + a_3\theta^3 + a_4\theta^4) \bmod q$ 
4:  $z_2 \leftarrow (a_0 + a_1\theta^2 + a_2\theta^4 + a_3\theta + a_4\theta^3) \bmod q$ 
5:  $z_3 \leftarrow (a_0 + a_1\theta^3 + a_2\theta + a_3\theta^4 + a_4\theta^2) \bmod q$ 
6:  $z_4 \leftarrow (a_0 + a_1\theta^4 + a_2\theta^3 + a_3\theta^2 + a_4\theta) \bmod q$ 
7: return  $\mathbf{z} = (z_0, z_1, z_2, z_3, z_4)$ 
```

---

---

**Algorithm 7**  $\text{NTT}_{128}(\mathbf{a}, \text{dir})$ 


---

**Input:**  $\mathbf{a} \in \mathbb{Z}_q^{128}$  and  $\text{dir} \in \{\text{FORWARD}, \text{INVERSE}\}$ .

**Output:**  $\mathbf{z}$ .

```

1:  $\tilde{\mathbf{a}} \leftarrow \text{bit\_reverse}(\mathbf{a})$ 
2: if  $\text{dir} = \text{FORWARD}$  then
3:    $\text{twiddle\_table} \leftarrow \{\omega_{128}^0, \omega_{128}^1, \dots, \omega_{128}^{127}\}$   $\triangleright \omega_{128}^{128} \bmod q = 1.$ 
4: else
5:    $\text{twiddle\_table} \leftarrow \{\omega_{128}^0, \omega_{128}^{-1}, \dots, \omega_{128}^{-127}\}$ 
6: end if
7: for ( $m \leftarrow 1; m \leq 7; m \leftarrow m + 1$ ) do
8:    $\text{step} \leftarrow 2^m; \text{half} \leftarrow 2^{m-1}; \text{stride} \leftarrow N_1/2^m;$ 
9:   for ( $i \leftarrow 0; i < 128; i \leftarrow i + \text{step}$ ) do
10:    for ( $j \leftarrow 0; j < \text{half}; j \leftarrow j + 1$ ) do
11:       $u \leftarrow \tilde{\mathbf{a}}[i + j + \text{half}] \cdot \text{twiddle\_table}[\text{stride} \cdot j] \bmod q$ 
12:       $\tilde{\mathbf{a}}[i + j + \text{half}] \leftarrow \tilde{\mathbf{a}}[i + j] + q - u$   $\triangleright \text{Lazy mod } q$ 
13:       $\tilde{\mathbf{a}}[i + j] \leftarrow \tilde{\mathbf{a}}[i + j] + u$   $\triangleright \text{Lazy mod } q$ 
14:    end for
15:  end for
16: end for
17:  $\mathbf{z}[i] \leftarrow \tilde{\mathbf{a}}[i] \bmod q, \quad i = 0, 1, \dots, 127$ 
18: return  $\mathbf{z}$ 

```

---

Let  $\mathbf{a} \in R_q$ . Then  $\text{NTT}_{640}(\mathbf{a}, \text{FORWARD})$  and  $\text{NTT}_{640}(\mathbf{a}, \text{INVERSE})$  denote the forward and inverse NTT transforms applied to the coefficient vector of  $\mathbf{a}$ , respectively. Let  $\mathbf{b} \in R_q^k$  be a polynomial vector. Then  $\text{NTT}_{640}(\mathbf{b}, \text{FORWARD})$  and  $\text{NTT}_{640}(\mathbf{b}, \text{INVERSE})$  denote the forward and inverse NTT transforms applied to each polynomial in  $\mathbf{b}$ , respectively. Let  $\mathbf{A} \in R_q^{k \times k}$ . Then  $\text{NTT}_{640}(\mathbf{A}, \text{FORWARD})$  and  $\text{NTT}_{640}(\mathbf{A}, \text{INVERSE})$  denote the forward and inverse NTT transforms applied to each polynomial entry in  $\mathbf{A}$ , respectively.

Let  $\mathbf{a} \in R_q$ ,  $\mathbf{b} \in R_q$ ,  $\hat{\mathbf{a}} = \text{NTT}_{640}(\mathbf{a}, \text{FORWARD}) = (\hat{a}_0, \hat{a}_1, \dots, \hat{a}_{N-1})$ , and  $\hat{\mathbf{b}} = \text{NTT}_{640}(\mathbf{b}, \text{FORWARD}) = (\hat{b}_0, \hat{b}_1, \dots, \hat{b}_{N-1})$ . We define

$$\hat{\mathbf{a}} \circ \hat{\mathbf{b}} = \text{NTT}_{640}((\hat{a}_0 \hat{b}_0, \hat{a}_1 \hat{b}_1, \dots, \hat{a}_{N-1} \hat{b}_{N-1}), \text{INVERSE}).$$

Let  $\mathbf{u} = (\mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{u}_{k-1}) \in R_q^k$ ,  $\mathbf{v} = (\mathbf{v}_0, \mathbf{v}_1, \dots, \mathbf{v}_{k-1}) \in R_q^k$ . Denote  $\hat{\mathbf{u}} = \text{NTT}_{640}(\mathbf{u}, \text{FORWARD})$ , and  $\hat{\mathbf{v}} = \text{NTT}_{640}(\mathbf{v}, \text{FORWARD})$ . We define

$$\hat{\mathbf{u}} \circ \hat{\mathbf{v}} = \sum_{i=0}^{k-1} \hat{\mathbf{u}}_i \circ \hat{\mathbf{v}}_i.$$

Let  $\mathbf{A} \in R_q^{k \times k}$ ,  $\mathbf{s} \in R_q^k$ ,  $\hat{\mathbf{A}} = \text{NTT}_{640}(\mathbf{A}, \text{FORWARD})$ , and  $\hat{\mathbf{s}} = \text{NTT}_{640}(\mathbf{s}, \text{FORWARD})$ . We define

$$\hat{\mathbf{A}} \circ \hat{\mathbf{s}} = (\hat{\mathbf{A}}_0 \circ \hat{\mathbf{s}}, \dots, \hat{\mathbf{A}}_{k-1} \circ \hat{\mathbf{s}}),$$

where  $\hat{\mathbf{A}}_i$  ( $i = 0, 1, \dots, k-1$ ) denotes the  $i$ -th row vector of  $\hat{\mathbf{A}}$ . Similarly, we define

$$\hat{\mathbf{s}} \circ \hat{\mathbf{A}} = (\hat{\mathbf{s}} \circ \hat{\mathbf{A}}_0, \dots, \hat{\mathbf{s}} \circ \hat{\mathbf{A}}_{k-1}),$$

where  $\hat{\mathbf{A}}_j$  ( $j = 0, 1, \dots, k-1$ ) denotes the  $j$ -th column vector of  $\hat{\mathbf{A}}$ .

**Vector Sampling over  $\mathbb{Z}_q^t$**  Algorithm 8 is a vector sampling algorithm for the vector space  $\mathbb{Z}_q^t$ . We implement this algorithm using a 16-bit rejection sampling method. The algorithm takes a byte array of length  $3t$  as input and outputs a vector  $\mathbf{a} = (a_0, a_1, \dots, a_{t-1}) \in \mathbb{Z}_q^t$ . For  $q = 7681$ , we set **threshold** =  $7681 \times 8 = 61448$ , and the single-trial rejection probability is  $1 - 61448/2^{16} = 0.0623779296875$ . When the input consists of  $3t$  bytes, the maximum number of trials is  $3t/2$ , and the sampling failure probability is

$$1 - \sum_{i=t}^{3t/2} \binom{3t/2}{i} \left(\frac{7681}{8192}\right)^i \left(\frac{511}{8192}\right)^{3t/2-i}.$$

In the Cheetah key encapsulation mechanism,  $t \geq N$ , so the sampling failure probability is much smaller than  $2^{-256}$ .

---

**Algorithm 8** SampleVec(**bytes**,  $t$ ): Vector sampling over  $\mathbb{Z}_q^t$ .

---

**Input:** Positive integer  $t$  and string **bytes**  $\in \mathbb{B}^{3t}$ .

**Output:**  $(a_0, a_1, \dots, a_{t-1}) \in \mathbb{Z}_q^t$ .

```

1:  $j \leftarrow 0$ 
2: for ( $i = 0; i < 3t$  and  $j < t; i = i + 2$ ) do
3:    $x \leftarrow \text{bytes}[i] + 2^8 \cdot \text{bytes}[i + 1]$ 
4:   if ( $x < \text{threshold}$ ) then
5:      $a_j \leftarrow x \bmod q$ 
6:      $j \leftarrow j + 1$ 
7:   end if
8: end for
9: return  $(a_0, a_1, \dots, a_{t-1})$ 

```

---

**Polynomial Sampling** Algorithm 9 samples a polynomial over  $\mathcal{R}_q$ . It takes a byte array of length  $3N$  as input and outputs the coefficient vector  $(f_0, f_1, \dots, f_{N-1})$  of a polynomial  $\mathbf{f} \in \mathcal{R}_q$ .

---

**Algorithm 9** SamplePoly(**bytes**): Polynomial sampling.

---

**Input:** String **bytes**  $\in \mathbb{B}^{3N}$ .

**Output:**  $\mathbf{f} = (f_0, f_1, \dots, f_{N-1})$ .

```

1:  $\mathbf{f} \leftarrow \text{SampleVec}(\text{bytes}, N)$ 
2: return  $\mathbf{f}$ 

```

---

**Polynomial Matrix Sampling** Let the polynomial matrix be

$$\mathbf{B} = \begin{pmatrix} \mathbf{b}_{00} & \mathbf{b}_{01} & \cdots & \mathbf{b}_{0,v-1} \\ \mathbf{b}_{10} & \mathbf{b}_{11} & \cdots & \mathbf{b}_{1,v-1} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{b}_{u-1,0} & \mathbf{b}_{u-1,1} & \cdots & \mathbf{b}_{u-1,v-1} \end{pmatrix},$$

where  $\mathbf{b}_{ij} \in R_q$  for  $0 \leq i < u$ ,  $0 \leq j < v$ . We arrange the coefficients of the polynomials into a one-dimensional vector:

$$\mathbf{b} = (\mathbf{b}_{00}, \dots, \mathbf{b}_{0,v-1}, \mathbf{b}_{1,0}, \dots, \mathbf{b}_{1,v-1}, \dots, \mathbf{b}_{u-1,v-1}) \in \mathbb{Z}_q^{uvN}.$$

The polynomial matrix sampling algorithm is given as follows:

---

**Algorithm 10** SamplePolyMat(**bytes**,  $u, v$ ): Polynomial matrix sampling.

---

**Input:** Positive integers  $u$  and  $v$ , string **bytes**  $\in \mathbb{B}^{3uvN}$ .

**Output:**  $\mathbf{b} = (\mathbf{b}_{00}, \dots, \mathbf{b}_{0,v-1}, \mathbf{b}_{1,0}, \dots, \mathbf{b}_{1,v-1}, \dots, \mathbf{b}_{u-1,v-1}) \in \mathbb{Z}_q^{uvN}$ .

1:  $\mathbf{b} \leftarrow \text{SampleVec}(\text{bytes}, uvN)$

2: **return**  $\mathbf{b}$

---

**Noise Vector Sampling** The coefficients of the noise polynomials in the Cheetah key encapsulation mechanism are sampled according to the centered binomial distribution. Let  $\eta$  be a positive integer. Algorithm 11 presents the noise vector sampling algorithm. The input of the algorithm is a byte array  $\mathbf{b}$  of length  $2t\eta/8$ , and the output is a noise vector  $\mathbf{e} \in \mathbb{Z}_q^t$ .

---

**Algorithm 11** SampleNoiseVec( $\mathbf{b}, t$ ): Noise polynomial sampling.

---

**Input:** Positive integer  $t$  and byte array  $\mathbf{b} \in \mathbb{B}^{2t\eta/8}$ .

**Output:** Noise vector  $\mathbf{e} \in \mathbb{Z}_q^t$ .

1:  $(b_0, b_1, \dots, b_{2t\eta-1}) \leftarrow \text{BytesToBits}(\mathbf{b})$

2: **for**  $(i = 0; i < t; i++)$  **do**

3:    $a \leftarrow \sum_{j=0}^{\eta-1} b_{2i\eta+j}$

4:    $b \leftarrow \sum_{j=0}^{\eta-1} b_{2i\eta+\eta+j}$

5:    $e_i \leftarrow a - b$

6: **end for**

7: **return**  $\mathbf{e} = (e_0, e_1, \dots, e_{t-1})$ .

---

**Noise Polynomial Vector Sampling** Let  $\mathbf{e} = (\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_k) \in \mathcal{R}^k$  be a vector of noise polynomials, where the coefficients of these vectors follow the centered binomial distribution. The sampling algorithm for the noise polynomial vector is shown in Algorithm 12. The input of the algorithm is a byte array **bytes**, and the output is a vector composed of the coefficients of the noise polynomial vector  $\mathbf{e}$ .

---

**Algorithm 12** `SampleNoisePolyVec(bytes, k)`: Noise polynomial sampling.

---

**Input:** String `bytes`  $\in \mathbb{B}^{2kN\eta/8}$ .

**Output:**  $\mathbf{e} = (e_{00}, \dots, e_{0,N-1}, e_{1,0}, \dots, e_{1,N-1}, \dots, e_{k-1,N-1}) \in \mathbb{Z}_q^{kN}$ .

1:  $\mathbf{e} \leftarrow \text{SampleNoiseVec}(\text{bytes}, kN)$

2: **return**  $\mathbf{e}$

---

**Encoding and Decoding of Polynomial Vectors** Let  $\mathbf{b} = (\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{k-1}) \in R^k$  be a polynomial vector whose coefficients lie in the interval  $[0, 2^d - 1]$ . We regard the coefficients of the polynomial vector as an element in the vector space  $\mathbb{Z}^{kN}$ . The encoding algorithm for  $\mathbf{b}$  is given in Algorithm 13.

---

**Algorithm 13** `EncodePolyVec(b, k, d)`: Polynomial vector encoding.

---

**Input:** Polynomial vector  $\mathbf{b} \in R^k$ , positive integers  $k$  and  $d$ .

**Output:** Byte array `bytes`  $\in \mathbb{B}^{kNd/8}$ .

1: `bytes`  $\leftarrow \text{IntegersToBytes}(\mathbf{b}, kN, d)$

2: **return** `bytes`

---

The corresponding decoding algorithm is given in Algorithm 14.

---

**Algorithm 14** `DecodePolyVec(bytes, k, d)`: Polynomial vector decoding.

---

**Input:** Byte array `bytes`  $\in \mathbb{B}^{kNd/8}$ , positive integers  $k$  and  $d$ .

**Output:**  $\mathbf{b} \in R^k$ .

1:  $(\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{k-1}) \leftarrow \text{BytesToIntegers}(\text{bytes}, kN, d)$

2: **return**  $(\mathbf{b}_0, \mathbf{b}_1, \dots, \mathbf{b}_{k-1})$

---

**Auxiliary Cryptographic Functions** The Cheetah key encapsulation mechanism requires a cryptographically secure pseudorandom function PRF, hash functions  $\mathcal{H}$ ,  $\mathcal{G}$ , and an extendable-output function XOF.

## 2.2 Cheetah Public-Key Encryption Algorithm

This subsection presents the CPA-secure Cheetah public-key encryption algorithm.

**Public and Secret Key Generation** Algorithm 15 is the key generation algorithm for the Cheetah PKE, where  $\lambda$  denotes the security parameter. For example,  $\lambda = 256$  when the cryptographic scheme provides 256 bits of classical security. The algorithm outputs a public key and a private key.  $\mathbf{s}$  and  $\mathbf{e}$  are noise polynomial vectors, whose coefficients each follow the centered binomial distribution. We assume that the matrix  $\hat{\mathbf{A}}$  in Step 4 of the algorithm is already in the NTT domain rather than an ordinary polynomial matrix, which reduces the number of NTT computations.

---

**Algorithm 15** `Cheetah_PKE_KeyGen()`: Cheetah key generation algorithm

---

**Input:** NULL.

**Output:** Public key  $\mathbf{pk}$  and private key  $\mathbf{sk}$ .

```

1:  $seed \in \mathbb{B}^{\lambda/8+16} \leftarrow \text{RNG}()$ 
2:  $(\rho, \sigma, \gamma) \in \mathbb{B}^{\lambda/8+16} \times \mathbb{B}^{\lambda/8+16} \times \mathbb{B}^{\lambda/8+16} \leftarrow \text{XOF}(seed)$ 
3:  $\mathbf{x} \leftarrow \text{XOF}(\rho)$   $\triangleright |\mathbf{x}| = 3k^2N.$ 
4:  $\hat{\mathbf{A}} \leftarrow \text{SamplePolyMat}(\mathbf{x}, k, k)$   $\triangleright \hat{\mathbf{A}} \in R_q^{k \times k}.$ 
5:  $\mathbf{y} \leftarrow \text{XOF}(\sigma)$   $\triangleright |\mathbf{y}| = 2kN\eta/8.$ 
6:  $\mathbf{s} \leftarrow \text{SampleNoisePolyVec}(\mathbf{y}, k)$   $\triangleright \mathbf{s} \in R_q^k.$ 
7:  $\mathbf{z} \leftarrow \text{XOF}(\gamma)$   $\triangleright |\mathbf{z}| = 2kN\eta/8.$ 
8:  $\mathbf{e} \leftarrow \text{SampleNoisePolyVec}(\mathbf{z}, k)$   $\triangleright \mathbf{e} \in R_q^k.$ 
9:  $\hat{\mathbf{s}} \leftarrow \text{NTT}_{640}(\mathbf{s}, \text{FORWARD})$ 
10:  $\mathbf{b} \leftarrow \text{Compress}(\hat{\mathbf{A}} \circ \hat{\mathbf{s}} + \mathbf{e}, 2^{\lceil \log_2(q) \rceil - d_b})$ 
11:  $\mathbf{pk} \leftarrow \rho \parallel \text{EncodePolyVec}(\mathbf{b}, k, d_b)$ 
12:  $\mathbf{sk} \leftarrow \text{EncodePolyVec}(\hat{\mathbf{s}}, k, \lceil \log_2(q) \rceil)$ 
13: return  $(\mathbf{pk}, \mathbf{sk})$ 
```

---

**Encryption** Algorithm 16 is the Cheetah encryption algorithm. We first encode the  $\lambda$ -bit plaintext into a binary polynomial  $\mathbf{m}$ , and then perform encryption. In Step 6, we hide part of the coefficients of  $\tilde{\mathbf{v}}$  and only the first  $\lambda$  coefficients of  $\tilde{\mathbf{v}}$  to be send. This does not affect the security of the algorithm and reduces the ciphertext size. From a security perspective, by concealing  $N - \lambda$  coefficients of the polynomial  $\tilde{\mathbf{v}}$ , the number of LWE samples available to the adversary is reduced by  $N - \lambda$ , which increases the difficulty of cryptanalysis.

---

**Algorithm 16** Cheetah\_PKE\_Encrypt(pk, m, γ): Cheetah Encryption Algorithm

---

**Input:** Public key pk, plaintext  $\mathbf{m} \in R_q$ , and seed string  $\gamma = (\gamma_1, \gamma_2, \gamma_3) \in \mathbb{B}^{\lambda/8+16} \times \mathbb{B}^{\lambda/8+16} \times \mathbb{B}^{\lambda/8+16}$ , where  $\mathbf{m}$  is a binary polynomial.

**Output:** Ciphertext  $(\mathbf{c}_1, \mathbf{c}_2)$ .

```

1:  $\rho \leftarrow \text{pk}[0 : \lambda/8 + 16 - 1]$ 
2:  $\mathbf{x} \leftarrow \text{XOF}(\rho)$   $\triangleright |\mathbf{x}| = 3k^2N$ .
3:  $\hat{\mathbf{A}} \leftarrow \text{SamplePolyMat}(\mathbf{x}, k, k)$   $\triangleright \hat{\mathbf{A}} \in R_q^{k \times k}$ .
4:  $\tilde{\mathbf{b}} \leftarrow \text{DecodePolyVec}(\text{pk}[\lambda/8 + 16 : \lambda/8 + 16 + kNd_b/8 - 1], k, d_b)$ 
5:  $\mathbf{b} \leftarrow \text{Decompress}(\tilde{\mathbf{b}}, 2^{\lceil \log_2(q) \rceil - d_b})$ 
6:  $\mathbf{y} \leftarrow \text{XOF}(\gamma_1)$   $\triangleright |\mathbf{y}| = 2kN\eta/8$ .
7:  $\mathbf{r} \leftarrow \text{SampleNoisePolyVec}(\mathbf{y}, k)$   $\triangleright \mathbf{r} \in R_q^k$ .
8:  $\mathbf{z} \leftarrow \text{XOF}(\gamma_2)$   $\triangleright |\mathbf{z}| = 2kN\eta/8$ .
9:  $\mathbf{e}_1 \leftarrow \text{SampleNoisePolyVec}(\mathbf{z}, k)$   $\triangleright \mathbf{e}_1 \in R_q^k$ .
10:  $\mathbf{w} \leftarrow \text{XOF}(\gamma_3)$   $\triangleright |\mathbf{w}| = 2N\eta/8$ .
11:  $\mathbf{e}_2 \leftarrow \text{SampleNoisePolyVec}(\mathbf{w}, 1)$   $\triangleright \mathbf{e}_2 \in R_q$ .
12:  $\hat{\mathbf{r}} \leftarrow \text{NTT}_{640}(\mathbf{r}, \text{FORWARD})$   $\triangleright \hat{\mathbf{r}} \in R_q^k$ .
13:  $\mathbf{u} \leftarrow \text{Compress}(\hat{\mathbf{r}} \circ \hat{\mathbf{A}} + \mathbf{e}_1, 2^{\lceil \log_2(q) \rceil - d_u})$ 
14:  $\hat{\mathbf{b}} \leftarrow \text{NTT}_{640}(\mathbf{b}, \text{FORWARD})$ 
15:  $\tilde{\mathbf{v}} \leftarrow \text{Truncate}(\hat{\mathbf{r}} \circ \hat{\mathbf{b}} + \mathbf{e}_2, \lambda) + \Delta \cdot \mathbf{m}$ 
16:  $\mathbf{v} \leftarrow \text{Compress}(\tilde{\mathbf{v}}, 2^{\lceil \log_2(q) \rceil - d_v})$ 
17:  $\mathbf{c}_1 \leftarrow \text{EncodePolyVec}(\mathbf{u}, k, d_u)$ 
18:  $\mathbf{c}_2 \leftarrow \text{IntegersToBytes}(\mathbf{v}, \lambda, d_v)$ 
19: return  $(\mathbf{c}_1, \mathbf{c}_2)$ .
```

---

**Decryption** Algorithm 17 is the Cheetah decryption algorithm. The decryption process is similar to that of NTRU and is very efficient.

---

**Algorithm 17** Cheetah\_PKE\_Decrypt(sk, c<sub>1</sub>, c<sub>2</sub>): Cheetah Decryption Algorithm

---

**Input:** Private key sk and ciphertext  $(\mathbf{c}_1, \mathbf{c}_2)$ .

**Output:** Plaintext  $\mathbf{m}$ .

```

1:  $\mathbf{u} \leftarrow \text{DecodePolyVec}(\mathbf{c}_1, k, d_u)$ 
2:  $\mathbf{v} \leftarrow \text{BytesToIntegers}(\mathbf{c}_2, \lambda, d_v)$ 
3:  $\hat{\mathbf{s}} \leftarrow \text{DecodePolyVec}(\text{sk}, k, \lceil \log_2(q) \rceil)$ 
4:  $\mathbf{u}' \leftarrow \text{Decompress}(\mathbf{u}, 2^{\lceil \log_2(q) \rceil - d_u})$ 
5:  $\mathbf{v}' \leftarrow \text{Decompress}(\mathbf{v}, 2^{\lceil \log_2(q) \rceil - d_v})$ 
6:  $\hat{\mathbf{u}} \leftarrow \text{NTT}_{640}(\mathbf{u}', \text{FORWARD})$ 
7:  $\mathbf{m}' \leftarrow (2 \cdot (\mathbf{v}' - \text{Truncate}(\hat{\mathbf{u}} \circ \hat{\mathbf{s}}, \lambda)) \bmod^{\pm} q) \bmod 2$ 
8:  $\mathbf{m} \leftarrow \text{IntegersToBytes}(\mathbf{m}', \lambda, 1)$ 
9: return  $\mathbf{m}$ .
```

---

**Correctness** Notice that

$$\begin{aligned}
& 2 \cdot (\mathbf{v}' - \text{Truncate}(\widehat{\mathbf{u}} \circ \widehat{\mathbf{s}}, \lambda)) \mod^{\pm q} \\
&= 2 \cdot (\text{Decompress}(\mathbf{v}, 2^{d_q-d_v}) - \text{Truncate}(\mathbf{s} \cdot \mathbf{u}', \lambda)) \mod^{\pm q} \\
&= 2 \cdot (\text{Decompress}(\text{Truncate}(\widetilde{\mathbf{v}}, \lambda), 2^{d_q-d_v}) - \text{Truncate}(\mathbf{s} \cdot \mathbf{u}', \lambda)) \mod^{\pm q} \\
&= 2 \cdot (\text{Truncate}(\text{Decompress}(\widetilde{\mathbf{v}}, 2^{d_q-d_v}), \lambda) - \text{Truncate}(\mathbf{s} \cdot \text{Decompress}(\mathbf{u}, 2^{d_q-d_u}), \lambda)) \mod^{\pm q} \\
&= \text{Truncate}(2 \cdot (\text{Decompress}(\widetilde{\mathbf{v}}, 2^{d_q-d_v}) - \mathbf{s} \cdot \text{Decompress}(\mathbf{u}, 2^{d_q-d_u})), \lambda) \mod^{\pm q} \\
&= \text{Truncate}(2 \cdot (\text{Decompress}(\text{Compress}(\mathbf{b} \cdot \mathbf{r} + \mathbf{e}_2 + c \cdot \mathbf{m}, 2^{d_q-d_v}), 2^{d_q-d_v}) \\
&\quad - \mathbf{s} \cdot \text{Decompress}(\text{Compress}(\mathbf{a} \cdot \mathbf{r} + \mathbf{e}_1, 2^{d_q-d_u}), 2^{d_q-d_u})), \lambda) \mod^{\pm q} \\
&= \text{Truncate}(2 \cdot (\mathbf{b} \cdot \mathbf{r} + \mathbf{e}_2 + \mathbf{e}' - \mathbf{s} \cdot (\mathbf{a} \cdot \mathbf{r} + \mathbf{e}_1 + \mathbf{e}'') + \mathbf{m}), \lambda) \mod^{\pm q} \\
&= \text{Truncate}(2(\mathbf{e} \cdot \mathbf{r} + \mathbf{e}_2 + \mathbf{e}' - \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}'') + \mathbf{m}, \lambda) \mod^{\pm q},
\end{aligned}$$

where  $\|\mathbf{e}'\|_{\infty} \leq 2^{d_q-d_v-1}$  and  $\|\mathbf{e}''\|_{\infty} \leq 2^{d_q-d_u-1}$ . If

$$\|\text{Truncate}(2(\mathbf{e} \cdot \mathbf{r} + \mathbf{e}_2 + \mathbf{e}' - \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}'') + \mathbf{m}, \ell)\|_{\infty} < q/2,$$

then

$$\begin{aligned}
& \text{Truncate}(2(\mathbf{e} \cdot \mathbf{r} + \mathbf{e}_2 + \mathbf{e}' - \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}'') + \mathbf{m}, \lambda) \mod^{\pm q} \\
&= \text{Truncate}(2(\mathbf{e} \cdot \mathbf{r} + \mathbf{e}_2 + \mathbf{e}' - \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}'') + \mathbf{m}, \lambda),
\end{aligned}$$

and we can successfully decrypt the ciphertext. Furthermore, since

$$\|\text{Truncate}(2(\mathbf{e} \cdot \mathbf{r} + \mathbf{e}_2 + \mathbf{e}' - \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}'') + \mathbf{m}, \ell)\|_{\infty} \leq \|2(\mathbf{e} \cdot \mathbf{r} + \mathbf{e}_2 + \mathbf{e}' - \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}'') + \mathbf{m}\|_{\infty},$$

it follows that if

$$\|2(\mathbf{e} \cdot \mathbf{r} + \mathbf{e}_2 + \mathbf{e}' - \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}'') + \mathbf{m}\|_{\infty} < q/2,$$

then the ciphertext can be decrypted successfully.

**Probability of Decryption Failure** Decryption failure may occur when

$$\|2(\mathbf{e} \cdot \mathbf{r} + \mathbf{e}_2 + \mathbf{e}' - \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}'') + \mathbf{m}\|_{\infty} \geq \frac{q}{2}.$$

We now proceed to compute the probability of decryption failure. On one hand, the coefficients of polynomials  $\mathbf{s}, \mathbf{e}, \mathbf{r}, \mathbf{e}_1, \mathbf{e}_2$  follow the centered binomial distribution  $\psi_{\eta}$ . On the other hand, we choose an appropriate  $d_u$  such that all coefficients of  $\mathbf{e}''$  lie in the set  $\{-\eta, \dots, -1, 0, 1, \dots, \eta\}$ . Under this setup, decryption fails if

$$\|\mathbf{e} \cdot \mathbf{r} + \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}''\|_{\infty} \geq \frac{q}{4} - 2^{d_q-d_v-1} - \eta.$$

Suppose the coefficients of  $\mathbf{e}, \mathbf{r}, \mathbf{e}_1, \mathbf{s}, \mathbf{e}''$  follow a Gaussian distribution with zero mean and standard deviation  $\sigma$ . Then each coefficient of  $\mathbf{e} \cdot \mathbf{r} + \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}''$  approximately obeys a Gaussian distribution with zero mean and standard deviation  $\sqrt{3n}\sigma^2$ . Accordingly, the probability that

$$\|\mathbf{e} \cdot \mathbf{r} + \mathbf{s} \cdot \mathbf{e}_1 - \mathbf{s} \cdot \mathbf{e}''\|_{\infty} \geq \frac{q}{4} - 2^{d_q-d_v-1} - \eta$$

is approximately

$$1 - \text{erf} \left( \frac{q/4 - 2^{d_q - d_v - 1} - \eta}{\sqrt{2} \cdot \sqrt{3n} \sigma^2} \right),$$

where  $\text{erf}(z) = \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2} dt$  and  $\sigma^2 = \eta/2$ . Therefore, the overall probability of decryption failure is approximately

$$n \cdot \left( 1 - \text{erf} \left( \frac{q/4 - 2^{d_q - d_v - 1} - \eta}{\sqrt{2} \cdot \sqrt{3n} \sigma^2} \right) \right).$$

### 2.3 Cheetah Key Encapsulation Mechanism

We convert the Cheetah public-key encryption algorithm into a key encapsulation mechanism using the Fujisaki-Okamoto transform [1]. Algorithm 18 is the key generation algorithm, where  $ek_{size} = \lambda/8 + kNd_b/8 + 16$  and  $dk_{size} = kN \lceil \log_2 q \rceil / 8 + ek_{size} + |H(ek)| + \lambda/8 + 16$ .

---

#### Algorithm 18 Cheetah\_KEM\_KeyGen()

---

**Input:** NULL

**Output:**  $ek \in \mathbb{B}^{ek_{size}}$  and  $dk \in \mathbb{B}^{dk_{size}}$

- 1:  $salt \leftarrow \mathbb{B}^{\lambda/8+16}$
  - 2:  $(ek_{PKE}, dk_{PKE}) \leftarrow \text{Cheetah\_PKE\_KeyGen}()$  ▷ Execute Algorithm 15.
  - 3:  $ek \leftarrow ek_{PKE}$
  - 4:  $dk \leftarrow dk_{PKE} \| ek \| H(ek) \| salt$
  - 5: **return**  $(ek, dk)$
- 

---

#### Algorithm 19 Cheetah\_KEM\_Encaps( $ek, m$ )

---

**Input:** Encapsulation key  $ek \in \mathbb{B}^{ek_{size}}$  and random string  $m \in \mathbb{B}^{\lambda/8}$ .

**Output:** Shared secret key  $K \in \mathbb{B}^{\lambda/8}$  and ciphertext  $c \in \mathbb{B}^{(kNd_u + \lambda d_v)/8}$ .

- 1:  $m \leftarrow \mathbb{B}^{\lambda/8}$
  - 2:  $(K, \gamma) \leftarrow G(m \| H(ek))$
  - 3:  $c \leftarrow \text{Cheetah\_PKE\_Encrypt}(ek, m, \gamma)$
  - 4: **return**  $(K, c)$
-

---

**Algorithm 20** Cheetah\_KEM\_Decaps( $dk, \mathbf{c}$ )

---

**Input:** Decapsulation key  $dk \in \mathbb{B}^{dk_{size}}$  and ciphertext  $\mathbf{c} \in \mathbb{B}^{(kNd_u + \lambda d_v)/8}$ .

**Output:** Shared secret key  $K' \in \mathbb{B}^{\lambda/8}$ .

- 1:  $(\mathbf{dk}_{PKE}, \mathbf{ek}_{PKE}, \mathbf{h}, salt) \leftarrow dk$
  - 2:  $\mathbf{m}' \leftarrow \text{Cheetah\_PKE\_Decrypt}(\mathbf{dk}_{PKE}, \mathbf{c})$
  - 3:  $(K', \gamma') \leftarrow \mathbf{G}(\mathbf{m}' \parallel \mathbf{h})$
  - 4:  $\bar{K} \leftarrow \mathbf{H}(salt \parallel \mathbf{c})$
  - 5:  $\mathbf{c}' \leftarrow \text{Cheetah\_PKE\_Encrypt}(\mathbf{ek}_{PKE}, \mathbf{m}', \gamma')$
  - 6: **if**  $\mathbf{c} \neq \mathbf{c}'$  **then**
  - 7:      $K' \leftarrow \bar{K}$
  - 8: **end if**
  - 9: **return**  $K'$
- 

## 2.4 Parameters

After comprehensive consideration of security, performance, and decryption failure probability, we select the parameters for the Cheetah key encapsulation mechanism as shown in Table 2.

|            | $N$ | $k$ | $q$  | $\lambda$ | $\eta$ | $(d_b, d_u, d_v)$ | $\delta$   |
|------------|-----|-----|------|-----------|--------|-------------------|------------|
| Cheetah128 | 640 | 1   | 7681 | 128       | 5      | (10, 10, 4)       | $2^{-129}$ |
| Cheetah256 | 640 | 2   | 7681 | 256       | 4      | (10, 10, 4)       | $2^{-176}$ |
| Cheetah384 | 640 | 3   | 7681 | 384       | 3      | (11, 11, 4)       | $2^{-189}$ |
| Cheetah512 | 640 | 4   | 7681 | 512       | 2      | (11, 11, 8)       | $2^{-243}$ |

Table 2: Parameters of the Cheetah Key Encapsulation Mechanism

The parameter  $\lambda$  denotes the classical bit security strength of the key encapsulation mechanism. The corresponding encapsulation key lengths and security strengths for the Cheetah128, Cheetah256, Cheetah384, and Cheetah512 key encapsulation algorithms are 128 bits, 256 bits, 384 bits, and 512 bits, respectively. The parameter  $q = 7681$  enables fast polynomial multiplication using the NTT transform. The parameter  $N = 640$  is the degree of the modulus polynomial.  $k$  is the dimension of the public-key polynomial matrix.  $\eta$  is the parameter of the centered binomial distribution.  $d_b$  is the public-key compression parameter, and  $d_u$  and  $d_v$  are the ciphertext compression parameters.  $\delta$  denotes the decapsulation failure probability of the key encapsulation mechanism.

**Parameter Selection Explanation** The prime  $q = 7681$  is chosen for the following reason:  $7680 = 1280 \times 6$ , which makes  $q$  an NTT-friendly prime.

The parameter  $N = 640$  is selected for the reasons below:  $N = 640 = 5 \times 2^7$ . When  $q = 7681$ , polynomial multiplication can be computed via the NTT transform. In accordance with standard call requirements, the key encapsulation mechanism shall provide 512-bit security, which imposes a lower bound on  $N$ .

If  $N$  is too small (e.g.,  $N < 512$ ), we either need multiple polynomials to carry the 512-bit plaintext message, or a single polynomial whose coefficients each store multiple bits of message data. Taking computational performance into consideration, we set  $N = 640$ .

The parameters  $\eta, d_b, d_u, d_v$  are determined by the decryption failure probability formula and the security analysis formulas.

### 3 Design Principles

The main differences between the CheetahKEM key encapsulation algorithm and Kyber are listed as follows:

- Different decryption logic. The final step of the Kyber decryption algorithm compares the ciphertext value with  $q/4$ : if the value exceeds  $q/4$ , the corresponding plaintext bit is set to 1; otherwise, the plaintext bit is 0. In the Cheetah encryption algorithm, plaintext recovery only requires modular arithmetic operations. This procedure resembles the decryption process of the NTRU encryption scheme and improves decryption speed.
- Distinct compression and decompression methods for public keys and ciphertexts. CheetahKEM implements compression and decompression for elements in  $\mathbb{Z}_q$  solely via shift and addition operations, without division or modular reduction operations.
- Different parameter choices. To achieve 512-bit classical security, we set  $N = 640$ . To reduce ciphertext size, we employ the **Truncate** function to truncate ciphertext polynomials.

#### 3.1 The RLWE, MLWE and LWE problem

From a security perspective, the LWE problem possesses no algebraic structure, whereas the RLWE and MLWE problem exhibits algebraic structure. The RLWE and MLWE problems are special cases of the LWE problem. If an algorithm exists that solves the LWE problem, then under the same parameters and complexity, this algorithm can also solve the RLWE and MLWE problems.

From a performance perspective, cryptographic schemes based on the RLWE and MLWE problems are more efficient than those based on the LWE problem. They feature shorter key lengths and ciphertext sizes, as well as faster encryption and decryption speeds. Therefore, after comprehensively balancing the security and performance of the cryptographic scheme, we employ the RLWE and MLWE problems to design the Cheetah key encapsulation mechanism.

#### 3.2 Noise Distribution

In LWE-based cryptosystems, the following noise distributions are commonly used:

- Discrete Gaussian distribution, e.g., FrodoKEM[2].
- Centered binomial distribution, e.g., Kyber[3] and Dilithium[4].
- NTRU-type distribution, where polynomial coefficients belong to  $\{-1, 0, 1\}$  with fixed numbers of  $-1$ ,  $0$ , and  $1$ , e.g., the NTRU encryption scheme[5].
- Uniform distribution, e.g., Saber[6].

Since the centered binomial distribution can be efficiently implemented in constant time and provides better resistance against side-channel attacks, we use it to sample noise polynomials.

## 4 Performance Analysis

### 4.1 Performance

In our reference implementation, only the seed bits used to generate the public-key matrix  $\hat{\mathbf{A}}$  are stored for public and secret key storage, with a length of  $\lambda/8 + 16$  bytes. When bandwidth and storage space are not constrained, we may store the entire public-key matrix  $\hat{\mathbf{A}}$  to speed up key encapsulation and decapsulation.

The performance of the reference implementation and the AVX2 implementation of the Cheetah key encapsulation mechanism is shown in Table 3.

|                     |                                     |                                      |
|---------------------|-------------------------------------|--------------------------------------|
| Cheetah128          |                                     |                                      |
| <b>size</b> (Bytes) | <b>ref</b> (10 <sup>3</sup> Cycles) | <b>avx2</b> (10 <sup>3</sup> Cycles) |
| <b>sk</b> : 1936    | <b>gen</b> : 119                    | <b>gen</b> : 111                     |
| <b>pk</b> : 832     | <b>enc</b> : 152                    | <b>enc</b> : 146                     |
| <b>ct</b> : 864     | <b>dec</b> : 171                    | <b>dec</b> : 157                     |
| Cheetah256          |                                     |                                      |
| <b>size</b> (Bytes) | <b>ref</b> (10 <sup>3</sup> Cycles) | <b>avx2</b> (10 <sup>3</sup> Cycles) |
| <b>sk</b> : 3808    | <b>gen</b> : 308                    | <b>gen</b> : 289                     |
| <b>pk</b> : 1648    | <b>enc</b> : 351                    | <b>enc</b> : 330                     |
| <b>ct</b> : 1728    | <b>dec</b> : 388                    | <b>dec</b> : 356                     |
| Cheetah384          |                                     |                                      |
| <b>size</b> (Bytes) | <b>ref</b> (10 <sup>3</sup> Cycles) | <b>avx2</b> (10 <sup>3</sup> Cycles) |
| <b>sk</b> : 5920    | <b>gen</b> : 970                    | <b>gen</b> : 936                     |
| <b>pk</b> : 2704    | <b>enc</b> : 1030                   | <b>enc</b> : 998                     |
| <b>ct</b> : 2832    | <b>dec</b> : 1116                   | <b>dec</b> : 1068                    |
| Cheetah512          |                                     |                                      |
| <b>size</b> (Bytes) | <b>ref</b> (10 <sup>3</sup> Cycles) | <b>avx2</b> (10 <sup>3</sup> Cycles) |
| <b>sk</b> : 7872    | <b>gen</b> : 1568                   | <b>gen</b> : 1508                    |
| <b>pk</b> : 3600    | <b>enc</b> : 1630                   | <b>enc</b> : 1578                    |
| <b>ct</b> : 4032    | <b>dec</b> : 1753                   | <b>dec</b> : 1669                    |

Table 3: Performance of the Cheetah Key Encapsulation Mechanism

Test environment for the reference implementation and AVX2 implementation: CPU model: Intel Core i7-1065G7, CPU clock frequency: 1.30 GHz; system memory: 8 GB; operating system: Ubuntu 24.04.3 LTS with Linux kernel 6.14.0; compiler: gcc 13.3.0; compilation flags: `-O3 -march=native -fomit-frame-pointer`. The cycle counts in the table are the average values obtained by executing the corresponding functions 10,000 times. Neither the reference implementation nor the AVX2 implementation uses heap memory.

## 4.2 Test Vector

Using the auxiliary programs and data provided by the Commercial Cryptography Standard Research Institute, we have generated test vectors for all algorithm instances. These test vectors are located in the `Test_Vector` directory of the submitted archive, with the specific test vector files as follows:

```

/ Test_Vector/KAT_KEM_Cheetah128.txt
/ Test_Vector/KAT_KEM_Cheetah256.txt
/ Test_Vector/KAT_KEM_Cheetah384.txt
/ Test_Vector/KAT_KEM_Cheetah512.txt

```

## 5 Security Analysis

### 5.1 Security Proof

Assume that  $\mathsf{XOF}$ ,  $\mathsf{H}$  and  $\mathsf{G}$  are random oracles, the decision MLWE problem is hard, and the public key  $(\mathbf{A}, \mathbf{b})$  is uncompressed. We first prove that Cheetah-PKE achieves IND-CPA security.

**Theorem 1.** *If the decision MLWE problem is hard, then the Cheetah-PKE public-key encryption scheme is IND-CPA secure under chosen-plaintext attacks.*

*Proof.* Suppose there exists a probabilistic polynomial-time (PPT) adversary  $\mathcal{A}$  that wins the IND-CPA game with non-negligible advantage  $\epsilon$ . We construct a simulator  $\mathcal{B}$  to solve the decision MLWE problem. Assume  $\mathcal{B}$  receives a decision MLWE challenge sample  $(\mathbf{A}, \mathbf{b})$ , where  $\mathbf{b}$  is either drawn from the MLWE distribution, i.e.,  $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$ , or a uniformly random vector.  $\mathcal{B}$  directly forwards the challenge pair  $(\mathbf{A}, \mathbf{b})$  as the public key of Cheetah-PKE to  $\mathcal{A}$ . When  $\mathcal{A}$  submits two plaintexts  $\mathbf{m}_0, \mathbf{m}_1$ ,  $\mathcal{B}$  samples a random bit  $\beta \in \{0, 1\}$ .  $\mathcal{B}$  leverages the algebraic structure of the MLWE challenge to generate the ciphertext  $(\mathbf{u}, \mathbf{v})$ . If  $\mathbf{b}$  is a genuine MLWE sample, the distribution of ciphertext  $(\mathbf{u}, \mathbf{v})$  is identical to that output by Cheetah-PKE encrypting  $\mathbf{m}_\beta$ . If  $\mathbf{b}$  is uniformly random, the ciphertext is statistically independent of the plaintext  $\mathbf{m}_\beta$ . Thus, the advantage of  $\mathcal{A}$  in guessing  $\beta$  directly translates to the distinguishing advantage of  $\mathcal{B}$  against the decision MLWE problem, namely

$$Adv_{IND-CPA}^{Cheetah-PKE}(\mathcal{A}) \leq 2 \cdot Adv_{MLWE}(\mathcal{B}).$$

□

We apply the FujisakiOkamoto transform to convert the IND-CPA secure Cheetah-PKE scheme into the CheetahKEM key encapsulation mechanism. We then prove that CheetahKEM achieves tight IND-CCA security in the classical random oracle model and non-tight IND-CCA security in the quantum random oracle model. Following the results in [7], we state the following theorems.

**Theorem 2** (Theorem 3.1 and 3.2 in [7]). *For any adversary  $\mathcal{B}$  making at most  $q_D$  decryption failure queries,  $q_G$  queries to random oracle  $\mathsf{G}$ , and  $q_H$  queries to random oracle  $\mathsf{H}$ , there exists an adversary  $\mathcal{A}$  running in the same time complexity as  $\mathcal{B}$  such that*

$$Adv_{CheetahKEM}^{IND-CCA}(\mathcal{B}) \leq q_H \cdot \delta + \frac{q_H + 2q_G + 1}{|\mathcal{M}|} + 3 \cdot Adv_{Cheetah-PKE}^{IND-CPA}(\mathcal{A}).$$

**Theorem 3** (Theorem 4.4 and 4.6 in [7]). *For any quantum adversary  $\mathcal{B}$  making at most  $q_D$  classical decryption failure queries,  $q_G$  quantum queries to random oracle  $\mathsf{G}$ ,  $q_H$  quantum queries to random oracle  $\mathsf{H}$ , and  $q_{\hat{\mathsf{H}}}$  quantum queries to*

random oracle  $\widehat{H}$ , there exists a quantum adversary  $\mathcal{A}$  running in the same time complexity as  $\mathcal{B}$  such that

$$Adv_{CheetahKEM}^{IND-CCA}(\mathcal{B}) \leq (2q_{\widehat{H}} + q_H) \cdot \sqrt{8 \cdot \delta(q_G + 1)^2 + (1 + 2q_G) \sqrt{Adv_{CheetahPKE}^{IND-CPA}(\mathcal{A})}}.$$

## 5.2 Complexity of the BKZ Algorithm

Before evaluating the security of the Cheetah key encapsulation mechanism, we first introduce the computational complexity of the BKZ lattice reduction algorithm. The BKZ algorithm is currently the most effective algorithm for finding short lattice vectors. Within the BKZ algorithm, either the enumeration algorithm or the sieving algorithm is employed as a subroutine. In practical applications, the sieving algorithm achieves the best performance as a subroutine. Therefore, in this paper, we assume that the BKZ algorithm uses the sieving algorithm as its subroutine.

Let  $\beta$  denote the dimension of the sublattice on which the sieving algorithm is applied. According to [8, 9, 10, 11], on a classical computer, the time complexity of the sieving algorithm is  $2^{0.292\beta + o(\beta)}$ , and the space complexity is  $2^{0.2075\beta + o(\beta)}$ . According to [12, 13, 14, 15], on a quantum computer, the time complexity of the sieving algorithm is  $2^{0.257\beta + o(\beta)}$ , with a quantum space complexity of the same order. Since the main cost of the BKZ algorithm comes from the sieving algorithm, we take the computational complexity of the sieving algorithm as that of the BKZ algorithm.

Let  $d$  denote the dimension of the lattice  $\mathcal{L}$ . We next determine the value of  $\beta$ . According to [16, 17], the condition for the BKZ algorithm to successfully output the target short vector  $\mathbf{s}$  is

$$\sqrt{\beta/d} \cdot \|\mathbf{s}\| < \delta_\beta^{2\beta-d-1} \cdot \text{Vol}(\mathcal{L})^{1/d}, \quad (1)$$

where  $\delta_\beta$  is the root-Hermite factor of the BKZ algorithm. According to [18], for  $\beta > 50$ ,

$$\delta_\beta = \left( \frac{(\pi\beta)^{1/\beta}\beta}{2\pi e} \right)^{1/(2\beta-2)}.$$

We can then compute the minimal  $\beta$  ( $\beta > 50$ ) satisfying Inequality (1), and further evaluate the computational complexity of the BKZ algorithm.

## 5.3 Practical Security Evaluation of the Cheetah KEM

The Cheetah key encapsulation mechanism possesses certain algebraic structures. At present, we do not know how to efficiently exploit these algebraic structures to attack the cryptosystem, so we temporarily ignore such structures. Since the RLWE and MLWE problems can be regarded as variants of the original LWE problem, algorithms for solving the LWE problem can be used to solve the RLWE and MLWE problems.

Let the parameters of the Cheetah key encapsulation mechanism be  $(q, k, N)$ , and the public key be  $(\mathbf{A}, \mathbf{b})$ , where  $\mathbf{A} \in \mathbb{Z}^{m \times m}$ ,  $\mathbf{b} \in \mathbb{Z}^m$ , and  $m = kN$ .

### 5.3.1 Primal Attack

Construct the lattice

$$\mathcal{L}_q(\mathbf{A}, \mathbf{I}_m, -\mathbf{b}) = \{\mathbf{x} \in \mathbb{Z}^{2m+1} : (\mathbf{A}, \mathbf{I}_m, -\mathbf{b})\mathbf{x} \equiv \mathbf{0} \pmod{q}\}.$$

The dimension of the lattice is  $d = 2m + 1$ , and its volume is  $q^m$ . It is easy to verify that  $(\mathbf{s}, \mathbf{e}, 1)$  is a short vector of the lattice, so we can use the BKZ algorithm to find this short vector. The condition for the BKZ algorithm to successfully find this short vector is

$$\sigma\sqrt{\beta} < \delta_\beta^{2\beta-d-1} \cdot q^{m/d},$$

where  $\sigma$  is the standard deviation of the noise vector.

In the actual attack process, since the entries of vectors  $\mathbf{s}$  and  $\mathbf{e}$  are small, we can guess the values of some entries of  $\mathbf{s}$  or  $\mathbf{e}$  before applying the BKZ algorithm. This can slightly reduce the lattice dimension and thus lower the complexity of the BKZ algorithm. However, we cannot guess too many entries of  $\mathbf{s}$  or  $\mathbf{e}$ , otherwise the guessing complexity will be too high. We need to balance the number of guessed entries and the complexity of the BKZ algorithm. This is the so-called Hybrid Primal Attack.

### 5.3.2 Dual Attack

We first construct the dual lattice

$$\mathcal{L}_q(\mathbf{A}) = \{(\mathbf{v}, \mathbf{w}) \in \mathbb{Z}^{2m} : \mathbf{w}^\top \equiv \mathbf{v}^\top \mathbf{A} \pmod{q}\},$$

whose dimension is  $d = 2m$  and volume is  $q^m$ . Using the BKZ algorithm, we can find a short vector  $(\mathbf{v}, \mathbf{w})$  with length approximately  $\ell = \delta_\beta^{d-1} q^{1/2}$ . We use this short vector to construct a distinguisher to distinguish LWE samples from random samples. Given a vector  $\mathbf{b} \in \mathbb{Z}^m$ , compute

$$z \equiv \mathbf{v}^\top \mathbf{b} \equiv \mathbf{v}^\top \mathbf{A} \mathbf{s} + \mathbf{v}^\top \mathbf{e} \equiv \mathbf{w}^\top \mathbf{s} + \mathbf{v}^\top \mathbf{e} \pmod{q}.$$

If  $\mathbf{b} = \mathbf{A} \mathbf{s} + \mathbf{e}$ , then  $z$  follows a Gaussian distribution with standard deviation  $\ell\sigma$ . If  $\mathbf{b}$  is uniformly distributed, then  $z$  is also uniformly distributed. The statistical distance between the Gaussian distribution and the uniform distribution is  $\epsilon = 4 \exp(-2\pi^2\tau^2)$ , where  $\tau = \ell\sigma/q$ . Thus we need to generate  $1/\epsilon^2$  short vectors using the BKZ algorithm.

The sieving algorithm in BKZ generates  $2^{0.2075\beta}$  short vectors. Assuming all these short vectors have length at most  $\ell$ , we need to run the  $\epsilon$ -distinguisher  $R = \max\{1, 1/(2^{0.2075\beta}\epsilon^2)\}$  times to finally recover the secret key (or an equivalent key)  $\mathbf{s}$ .

Similar to the primal attack, in practice, since the entries of  $\mathbf{s}$  and  $\mathbf{e}$  are small, we can guess the values of some entries of  $\mathbf{s}$  or  $\mathbf{e}$  before applying the BKZ algorithm to reduce the attack complexity.

### 5.3.3 Attack Complexity of the Cheetah Key Encapsulation Mechanism

Table 4 shows the computational complexity of the primal attack and dual attack against the Cheetah key encapsulation mechanism.

| Algorithm  | Primal Attack |         | Dual Attack |         |
|------------|---------------|---------|-------------|---------|
|            | Classical     | Quantum | Classical   | Quantum |
| Cheetah128 | 159           | 139     | 157         | 137     |
| Cheetah256 | 307           | 276     | 302         | 269     |
| Cheetah384 | 426           | 382     | 418         | 375     |
| Cheetah512 | 541           | 497     | 522         | 482     |

Table 4: Computational complexity of the primal attack and dual attack on the Cheetah KEM. The values in the table are base-2 logarithms of the attack complexity.

## 6 Advantages and Disadvantages

The Cheetah KEM is easy to implement and extend. Implementations of the Cheetah KEM at different security levels differ only in macro definitions within header files, while other code can be reused. If the security level needs to be adjusted, it is only necessary to modify the macro definitions in the header files.

In the following, we illustrate the advantages and disadvantages of the Cheetah key encapsulation mechanism by comparing it with other post-quantum key encapsulation schemes.

| Scheme             | <b>sk</b>   | <b>pk</b>  | <b>ct</b>  | <b>gen</b> | <b>enc</b> | <b>dec</b> |
|--------------------|-------------|------------|------------|------------|------------|------------|
| <b>Cheetah128</b>  | <b>1936</b> | <b>832</b> | <b>864</b> | <b>119</b> | <b>152</b> | <b>171</b> |
| Kyber512[19]       | 1632        | 800        | 768        | 123        | 155        | 188        |
| FrodoKEM-640[2]    | 19888       | 9616       | 9720       | 1385       | 1862       | 1747       |
| NewHope512[20]     | 1888        | 928        | 1120       | 117        | 180        | 206        |
| ntruhs2048509[21]  | 935         | 699        | 699        | 8906       | 643        | 1662       |
| hqc-128[22]        | 40          | 2249       | 4497       | 180        | 408        | 875        |
| mceliece348864[23] | 6492        | 261120     | 128        | 58034      | 44         | 135        |

Table 5: Comparison of 128-bit secure key encapsulation algorithms, where the sizes of **sk**, **pk**, and **ct** are measured in Bytes; **gen**, **enc**, and **dec** are measured in  $10^3$  Cycles. All performance values are from reference implementations.

**Compared with Schemes Directly Based on the LWE Problem** There are two main approaches to constructing key encapsulation mechanisms or key

exchange protocols directly from the LWE problem. The first method is detailed in [24], which results in large public and secret keys but relatively small ciphertexts. The second, more practical approach is given in [2], namely the FrodoKEM key encapsulation mechanism. As shown in Table 5, the public key and ciphertext sizes of FrodoKEM-640 are approximately 11 times those of the Cheetah KEM. In terms of security, KEMs directly based on the LWE problem have no algebraic structure, whereas the Cheetah KEM possesses an algebraic structure and vulnerable to potential algebraic attacks.

**Compared with Schemes Based on the MLWE Problem** A typical key encapsulation mechanism based on the MLWE problem is the Kyber KEM [19], which is a post-quantum KEM standardized by NIST. In terms of security, Kyber512 uses the algebraic structure  $\mathbb{Z}_q[X]/(X^{256} + 1)$ , while Cheetah uses  $\mathbb{Z}_q[X]/(X^{640} + 1)$ . In terms of performance, Table 5 shows that Cheetah128 has slightly larger public key and ciphertext sizes than Kyber512, but Cheetah128 is faster encapsulation and decapsulation.

**Compared with Schemes Based on the RLWE Problem** A representative KEM based on the RLWE problem is the NewHope KEM [20]. In terms of security, NewHope512 uses the algebraic structure  $\mathbb{Z}_q[X]/(X^{512} + 1)$ , while Cheetah uses  $\mathbb{Z}_q[X]/(X^{640} + 1)$ . In terms of performance, Table 5 shows that Cheetah128 has smaller public key and ciphertext sizes than NewHope512, and is also faster in key generation, encapsulation, and decapsulation.

**Compared with NTRU** In [21], the ntruhs2048509 KEM uses the algebraic structure  $\mathbb{Z}_q[X]/(X^{509} - 1)$  with  $q = 2048$ . In terms of performance, Table 5 shows that Cheetah128 has slightly larger public key and ciphertext sizes than ntruhs2048509, but is faster in key generation, encapsulation, and decapsulation.

**Compared with Code-Based Schemes** Typical code-based key encapsulation mechanisms include Classic McEliece [23] and HQC [22]. As shown in Table 5, the combined total size of public keys and ciphertexts of these two schemes is larger than that of Cheetah128, and their speeds of key generation, encapsulation, and decapsulation are slower than Cheetah128.

## References

- [1] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In *Annual international cryptography conference*, pages 537–554. Springer, 1999.
- [2] Lewis Glabush, Patrick Longa, Michael Naehrig, Chris Peikert, Douglas Stebila, and Fernando Virdia. Frodokem: A cca-secure learning with errors key encapsulation mechanism. *Cryptology ePrint Archive*, 2025.

- [3] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber: a cca-secure module-lattice-based kem. In *2018 IEEE European symposium on security and privacy (EuroS&P)*, pages 353–367. IEEE, 2018.
- [4] Deepraj Soni, Kanad Basu, Mohammed Nabeel, Najwa Aaraj, Marc Manzano, Ramesh Karri, Deepraj Soni, Kanad Basu, Mohammed Nabeel, Najwa Aaraj, et al. Crystals-dilithium. *Hardware Architectures for Post-Quantum Digital Signature Schemes*, pages 13–30, 2021.
- [5] Jeffrey Hoffstein, Jill Pipher, and Joseph H Silverman. Ntru: A ring-based public key cryptosystem. In *International algorithmic number theory symposium*, pages 267–288. Springer, 1998.
- [6] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-lwr based key exchange, cpa-secure encryption and cca-secure kem. In *International Conference on Cryptology in Africa*, pages 282–305. Springer, 2018.
- [7] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the fujisaki-okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography*, pages 341–371, Cham, 2017. Springer International Publishing.
- [8] Thijs Laarhoven. Sieving for shortest vectors in lattices using angular locality-sensitive hashing. In *Annual Cryptology Conference*, pages 3–22. Springer, 2015.
- [9] Anja Becker and Thijs Laarhoven. Efficient (ideal) lattice sieving using cross-polytope lsh. In *International Conference on Cryptology in Africa*, pages 3–23. Springer, 2016.
- [10] Anja Becker, Léo Ducas, Nicolas Gama, and Thijs Laarhoven. New directions in nearest neighbor searching with applications to lattice sieving. In *Proceedings of the twenty-seventh annual ACM-SIAM symposium on Discrete algorithms*, pages 10–24. SIAM, 2016.
- [11] Martin R Albrecht, Léo Ducas, Gottfried Herold, Elena Kirshanova, Eamonn W Postlethwaite, and Marc Stevens. The general sieve kernel and new records in lattice reduction. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 717–746. Springer, 2019.
- [12] Thijs Laarhoven, Michele Mosca, and Van De Pol Joop. Finding shortest lattice vectors faster using quantum search. *Designs, Codes and Cryptography*, 77(2):375–400, 2015.

- [13] André Chailloux and Johanna Loyer. Lattice sieving via quantum random walks. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 63–91, 2021.
- [14] Max Heiser. Improved quantum hypercone locality sensitive filtering in lattice sieving. *Cryptology ePrint Archive*, 2021.
- [15] Divesh Aggarwal, Jianwei Li, Phong Q Nguyen, and Noah Stephens-Davidowitz. Slide reduction, revisited-filling the gaps in svp approximation. In *Annual International Cryptology Conference*, pages 274–295. Springer, 2020.
- [16] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange-a new hope. *Cryptology ePrint Archive*, 2015.
- [17] Tsuyoshi Takagi and Thomas Peyrin. Revisiting the expected cost of solving usvp and applications to lwe. *Springer, Cham*, 10.1007/978-3-319-70694-8(Chapter 11):297–322, 2017.
- [18] Yuanmi Chen and Phong Q. Nguyen. Bkz 2.0: Better lattice security estimates. *DBLP*, 2011.
- [19] Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-kyber algorithm specifications and supporting documentation. 2021.
- [20] Erdem Alkim, Roberto Avanzi, Joppe Bos, Léo Ducas, Antonio De La Piedra, Peter Schwabe Thomas Pöppelmann, and Douglas Stebila. Newhope. *Submission to round, 1*, 2017.
- [21] Cong Chen, Oussama Danba, Jeffrey Hoffstein, Andreas Hülsing, Joost Rijneveld, John M Schanck, Peter Schwabe, William Whyte, and Zhenfei Zhang. Algorithm specifications and supporting documentation. *Brown University and Onboard security company, Wilmington USA*, 2019.
- [22] Philippe Gaborit, Carlos Aguilar-Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Edoardo Persichetti, Gilles Zémor, Jurjen Bos, et al. Hamming quasi-cyclic (hqc). *NIST Post-Quantum Standardization, 4th Round; National Institute of Standards and Technology: Gaithersburg, MD, USA*, 2025.
- [23] Daniel J Bernstein, Tung Chou, Tanja Lange, Ingo von Maurich, Rafael Misoczki, Ruben Niederhagen, Edoardo Persichetti, Christiane Peters, Peter Schwabe, Nicolas Sendrier, et al. Classic mceliece: conservative code-based cryptography. *NIST submissions*, 1(1):1–25, 2017.
- [24] Chris Peikert, Vinod Vaikuntanathan, and Brent Waters. A framework for efficient and composable oblivious transfer. In *Annual international cryptology conference*, pages 554–571. Springer, 2008.